



Introduction to the Command Line

Bash & PowerShell Fundamentals

Chapter Overview

- 01** What is the Command Line?
- 02** Getting Started
- 03** Navigating the File System
- 04** Working with Files & Directories
- 05** Text Processing & Pipes
- 06** Environment & Configuration
- 07** Shell Scripting Basics
- 08** Class Exercises



What is the Command Line?

Section 1



The Command Line Interface (CLI)

Definition

- A text-based interface for interacting with your computer
- You type commands — the computer executes them and shows the result
- Also called: terminal, console, shell, command prompt

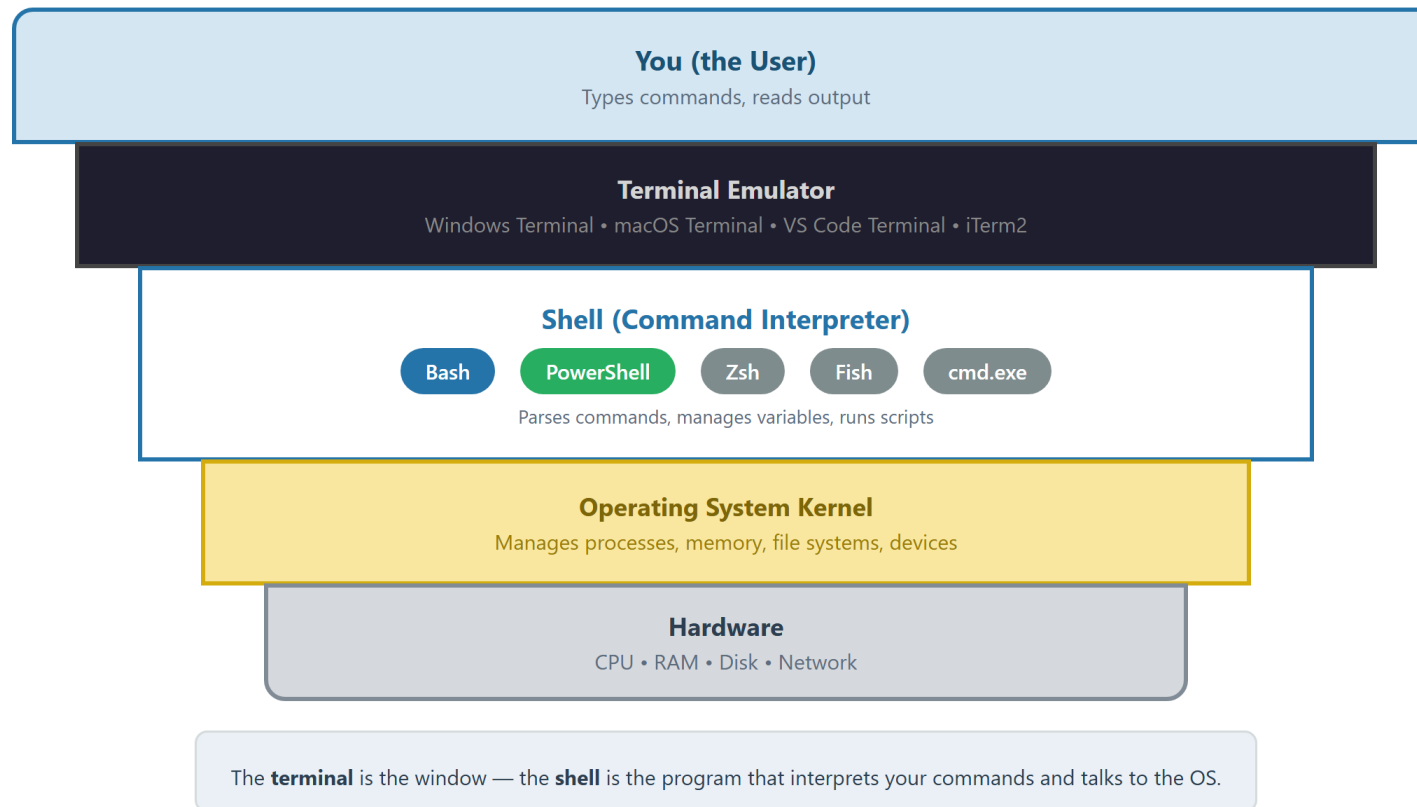
Why Learn the CLI?

- Many development tools are CLI-only (Git, npm, Docker, compilers)
- Faster than clicking through GUI menus once you know the commands
- Scriptable — automate repetitive tasks with shell scripts
- Works on servers — most production servers have no graphical interface



How the Command Line Works

How the Command Line Works



Bash vs PowerShell

Bash (Bourne Again Shell)

- Default shell on Linux and macOS
- Available on Windows via Git Bash or WSL
- Industry standard for DevOps, scripting, and server management

PowerShell

- Default on Windows — built into every Windows installation
- Cross-platform (PowerShell 7+ runs on Linux/macOS too)
- Object-oriented pipeline — passes structured data, not just text



Getting Started

Section 2



Opening a Terminal

Windows

- PowerShell: Search 'PowerShell' in Start menu, or Win+X → Terminal
- Git Bash: Install from git-scm.com — provides Bash on Windows
- VS Code: Ctrl+` opens the integrated terminal

macOS

- Terminal.app in Applications → Utilities (or Spotlight: 'terminal')
- Default shell is zsh (similar to Bash)

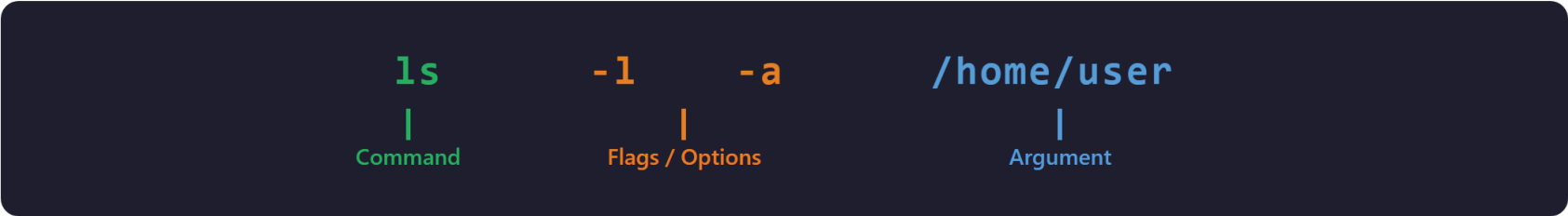
Linux

- Ctrl+Alt+T on most distributions
- Default shell is typically Bash



Anatomy of a Command

Anatomy of a Command



Bash Flag Styles	
<code>-l</code>	Short flag (single dash)
<code>-la</code>	Combined short flags
<code>--long-name</code>	Long flag (double dash)
<code>--output=file</code>	Flag with value

PowerShell Parameter Styles	
<code>-Recurse</code>	Named parameter (switch)
<code>-Path "C:\dir"</code>	Parameter with value
<code>-Filter *.txt</code>	Wildcard argument
<code> Select-Object</code>	Cmdlet (Verb-Noun)

Bash: `man ls` or `ls --help` **PowerShell:** `Get-Help Get-ChildItem` or `Get-ChildItem -?`



Your First Commands

```
# Bash / Git Bash
$ whoami          # Who am I logged in as?
$ date            # Current date and time
$ echo 'Hello World' # Print a message
$ clear           # Clear the terminal screen

# PowerShell
PS> whoami        # Same – works in both!
PS> Get-Date      # Current date and time
PS> Write-Host 'Hello World' # Print a message
PS> Clear-Host    # Clear screen (or cls)

# Both shells support Tab for autocomplete
# Both shells support Up/Down arrow for command history
```



Navigating the File System

Section 3



File System Navigation

File System Navigation

Directory Tree

```
/ (root)
├── -> home/
│   └── -> user/
│       ├── -> Documents/
│       ├── -> projects/
│       │   └── -> app.js
│       └── -> .bashrc
├── -> etc/
├── -> usr/
│   └── -> bin/
└── -> tmp/
```

Navigation Commands

<code>pwd</code>	Print current directory
<code>cd /path</code>	Change to directory
<code>cd ..</code>	Go up one level
<code>cd ~</code>	Go to home directory
<code>cd -</code>	Go to previous directory
<code>ls -la</code>	List all with details

Path Types

Absolute: `/home/user/projects`

Starts from root (`/`), always the same

Relative: `../Documents`

Starts from current directory

Navigation Commands

```
# Bash
$ pwd
$ ls
$ ls -la
$ cd Documents
$ cd ..
$ cd ~
$ cd -

# PowerShell
PS> Get-Location
PS> Get-ChildItem
PS> Get-ChildItem -Force
PS> Set-Location Documents
PS> Set-Location ..
PS> Set-Location ~
PS> Pop-Location

# Both shells understand:
# . = current directory
# .. = parent directory
# ~ = home directory
# Tab = autocomplete path names
```



Understanding Paths

Absolute Paths

- Start from the root — always point to the same location
- Bash: `/home/user/Documents` PowerShell: `C:\Users\user\Documents`

Relative Paths

- Start from your current directory
- `./src/app.js` — file 'app.js' inside 'src' folder from here
- `../../config` — go up two levels, then into 'config'

Path Separators

- Bash uses forward slashes: `/home/user/projects`
- PowerShell accepts both: `C:\Users` or `C:/Users`



Working with Files & Directories

Section 4



Creating Files & Directories

```
# Bash
$ mkdir myproject
$ mkdir -p a/b/c
$ touch index.html
```

```
# Create file with content
$ echo "Hello" > hello.txt
$ echo "More" >> hello.txt
```

```
# View file contents
$ cat hello.txt
$ head -5 largefile.txt
$ tail -10 logfile.txt
```

```
# PowerShell
PS> New-Item -Type Directory myproject
PS> New-Item -Type Directory a/b/c -Force
PS> New-Item index.html
```

```
PS> "Hello" | Out-File hello.txt
PS> "More" | Add-Content hello.txt
```

```
PS> Get-Content hello.txt
PS> Get-Content largefile.txt -First 5
PS> Get-Content logfile.txt -Tail 10
```


Copying, Moving & Deleting

# Bash	# PowerShell
# Copy a file	
\$ cp file.txt backup.txt	PS> Copy-Item file.txt backup.txt
# Copy a directory (recursive)	
\$ cp -r src/ backup/	PS> Copy-Item src/ backup/ -Recurse
# Move / rename	
\$ mv old.txt new.txt	PS> Move-Item old.txt new.txt
# Delete a file	
\$ rm file.txt	PS> Remove-Item file.txt
# Delete a directory (recursive)	
\$ rm -rf mydir/	PS> Remove-Item mydir -Recurse -Force

Wildcards & Globbing

Pattern Matching

- * matches any number of characters: *.txt matches all text files
- ? matches exactly one character: file?.txt matches file1.txt, fileA.txt
- [abc] matches any character in set: file[12].txt matches file1.txt, file2.txt

Practical Examples

- ls *.js — list all JavaScript files
- cp src/*.html dist/ — copy all HTML files to dist
- rm *.tmp — delete all temp files



Bash vs PowerShell — Common Commands

Bash vs PowerShell — Common Commands

Task	Bash	PowerShell
List files	<code>ls -la</code>	<code>Get-ChildItem (dir / ls)</code>
Change directory	<code>cd /path</code>	<code>Set-Location /path (cd)</code>
Create directory	<code>mkdir mydir</code>	<code>New-Item -Type Directory mydir</code>
Copy files	<code>cp source dest</code>	<code>Copy-Item source dest</code>
Move/rename	<code>mv old new</code>	<code>Move-Item old new</code>
Delete file	<code>rm file</code>	<code>Remove-Item file</code>
View file content	<code>cat file</code>	<code>Get-Content file (cat / type)</code>
Search text	<code>grep "pattern" file</code>	<code>Select-String "pattern" file</code>
Current directory	<code>pwd</code>	<code>Get-Location (pwd)</code>
Command history	<code>history</code>	<code>Get-History (history)</code>

PowerShell aliases in parentheses — e.g. `ls`, `cd`, `cat` work in both shells

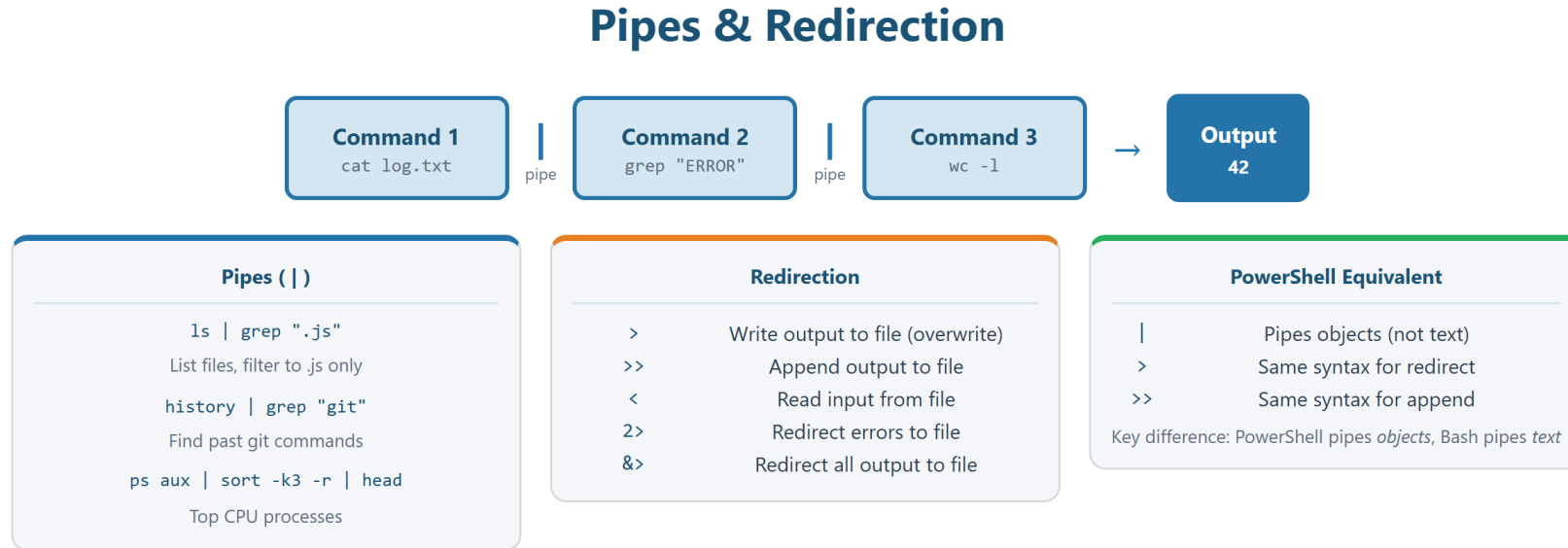


Text Processing & Pipes

Section 5



Pipes & Redirection



Searching and Filtering

```
# Bash: grep searches inside files
$ grep "error" logfile.txt          # Find lines with "error"
$ grep -i "error" logfile.txt       # Case-insensitive
$ grep -r "TODO" src/               # Search recursively in folder
$ grep -c "error" logfile.txt       # Count matching lines

# PowerShell: Select-String is PowerShell's grep
PS> Select-String "error" logfile.txt
PS> Select-String "error" logfile.txt -CaseSensitive
PS> Get-ChildItem src/ -Recurse | Select-String "TODO"

# Bash: find locates files
$ find . -name "*.js"               # Find all .js files
# PowerShell equivalent:
PS> Get-ChildItem -Recurse -Filter "*.js"
```



Piping Commands Together

```
# Bash: pipe text between commands with |  
$ cat server.log | grep "ERROR" | wc -l  
# Read file → filter errors → count lines
```

```
$ ls -la | sort -k5 -n -r | head -5  
# List files → sort by size → show top 5
```

```
$ history | grep "git" | tail -10  
# Command history → filter git commands → last 10
```

```
# PowerShell: pipe objects between cmdlets  
PS> Get-Process | Sort-Object CPU -Descending | Select-Object -First 5  
PS> Get-ChildItem | Where-Object { $_.Length -gt 1MB }  
PS> Get-Content log.txt | Where-Object { $_ -match 'ERROR' }
```



Environment & Configuration

Section 6



Variable Resolution: Bash vs PowerShell

Variable Resolution: Bash vs PowerShell

Bash Variable Resolution		PowerShell Variable Resolution	
Syntax	Resolves To	Syntax	Resolves To
<code>\$HOME</code>	<code>/home/username</code>	<code>\$env:USERPROFILE</code>	<code>C:\Users\username</code>
<code>\$PATH</code>	<code>/usr/local/bin:/usr/bin:/bin</code>	<code>\$env:PATH</code>	<code>C:\Windows;C:\Program Files...</code>
<code>\$USER</code>	<code>username</code>	<code>\$env:USERNAME</code>	<code>username</code>
<code>\$PWD</code>	<code>/home/username/project</code>	<code>\$PWD</code>	<code>C:\Users\username\project</code>
<code>"Hello \$USER"</code>	Hello username	<code>"Hello \$env:USERNAME"</code>	Hello username
<code>'Hello \$USER'</code>	Hello <code>\$USER</code> (literal)	<code>'Hello \$env:USERNAME'</code>	Hello <code>\$env:USERNAME</code> (literal)
<code>\$(command)</code>	output of command	<code>\$(command)</code>	output of command

Bash Key Rules

- `$VAR` — resolves the variable
- `"..."` — double quotes expand variables
- `'...'` — single quotes are **literal** (no expansion)
- Separator: colon `:` in PATH

PowerShell Key Rules

- `$env:VAR` — environment variables use **env:** prefix
- `"..."` — double quotes expand variables
- `'...'` — single quotes are **literal** (no expansion)
- Separator: semicolon `;` in PATH



Shell Configuration

Bash Configuration

- `~/.bashrc` — runs every time you open a new terminal
- Add aliases, custom PATH entries, and shell settings
- After editing, run: `source ~/.bashrc` to apply changes

PowerShell Profile

- `$PROFILE` — path to your PowerShell startup script
- Create if it doesn't exist: `New-Item -Force $PROFILE`
- Add aliases, functions, and module imports

Aliases

- Bash: `alias ll='ls -la'` PowerShell: `Set-Alias ll Get-ChildItem`



Shell Scripting Basics

Section 7



Why Write Shell Scripts?

Automation

- Run multiple commands in sequence with a single script
- Automate backups, deployments, testing, and setup tasks

Reproducibility

- Document your workflow as executable code
- Share scripts with teammates — everyone runs the same steps

Script Types

- Bash scripts: .sh files — run with `bash script.sh` or `./script.sh`
- PowerShell scripts: .ps1 files — run with `./script.ps1`



Bash Script Example

```
#!/bin/bash
# setup-project.sh – Create a new project structure

set -euo pipefail

PROJECT=$1                # First argument = project name

if [ -z "$PROJECT" ]; then
    echo "Usage: ./setup-project.sh <project-name>"
    exit 1
fi

mkdir -p "$PROJECT"/{src,tests,docs}
echo "# $PROJECT" > "$PROJECT/README.md"
touch "$PROJECT"/src/index.js

echo "Project $PROJECT created!"
ls -la "$PROJECT/"
```



PowerShell Script Example

```
# Setup-Project.ps1 – Create a new project structure

param(
    [Parameter(Mandatory)]
    [string]$ProjectName
)

foreach ($dir in @("src", "tests", "docs")) {
    New-Item -Type Directory -Path $ProjectName/$dir -Force
}

"# $ProjectName" | Out-File $ProjectName/README.md
New-Item $ProjectName/src/index.js

Write-Host "Project $ProjectName created!" -ForegroundColor Green
Get-ChildItem $ProjectName/
```



Class Exercises

Section 8



Exercise 1: Navigate the File System

Goal

- Practice navigating directories using the command line

Steps

- Open your terminal (Git Bash or PowerShell)
- Run `pwd` (or `Get-Location`) to see where you are
- Navigate to your home directory with `cd ~`
- Use `ls` (or `dir`) to list the contents
- Navigate into the Documents folder and back up with `cd ..`
- Use `ls -la` (or `Get-ChildItem -Force`) to view hidden files
- Use Tab to autocomplete directory names as you navigate



Exercise 2: Create a Project Structure

Goal

- Build a project folder structure entirely from the command line

Steps

- Create a directory called cli-practice with mkdir
- Inside it, create three folders: src, docs, tests
- Create a file called README.md with echo 'My Project' > README.md
- Create index.html inside src/ using touch (Bash) or New-Item (PowerShell)
- Copy README.md into the docs folder
- Rename docs/README.md to docs/guide.md using mv (or Move-Item)
- Verify the structure with ls -R (Bash) or Get-ChildItem -Recurse (PS)



Exercise 3: Search and Filter Data

Goal

- Practice grep/Select-String, pipes, and redirection

Steps

- Create a file called colors.txt with 10 colors (one per line)
- Use grep/Select-String to find a specific color in the file
- Count the lines with wc -l (Bash) or Measure-Object (PS)
- Use a pipe to sort the file: cat colors.txt | sort (or Get-Content | Sort-Object)
- Redirect sorted output to sorted_colors.txt using >
- Use history | grep (or Get-History | Select-String) to find past commands



Exercise 4: Write a Simple Script

Goal

- Create and run your first shell script

Steps (choose Bash or PowerShell)

- Create a file: greet.sh (Bash) or Greet.ps1 (PowerShell)
- Write a script that takes a name argument and prints 'Hello, <name>!'
- Bash: use \$1 for the argument, echo for output
- PowerShell: use param([string]\$Name), Write-Host for output
- Make the Bash script executable: chmod +x greet.sh
- Run it: ./greet.sh Alice (Bash) or ./Greet.ps1 -Name Alice (PS)
- Bonus: add an 'if' check for missing arguments



Key Takeaways

Key Takeaways

1. The terminal is the window — the shell (Bash/PowerShell) interprets commands
2. Navigation: `pwd`, `cd`, `ls` — the foundation of everything you do
3. File operations: `mkdir`, `cp`, `mv`, `rm` — manage your project from the CLI
4. Pipes connect commands — output of one becomes input of the next
5. Environment variables and `PATH` control how your system finds programs
6. Shell scripts automate repetitive tasks — learn once, run forever

